

PYTHON DAY 9 & DAY 10

Modules, Packages, Input & Output, Files, and JSON

Practical + Research-Oriented Learning

Total time: 5 to 7 hours | **Level:** Beginner → Intermediate

Method: Learn → Code → Predict → Break → Debug → Build → Research

Today's Big Idea

As Python programs become larger, keeping everything in one file becomes difficult. Modules and packages let you organize code into reusable parts. Input/output tools let your programs communicate with files and preserve information between runs.

The Mental Model

```
graph TD; A[Large program] --> B[Separate responsibilities]; B --> C[Modules]; C --> D[Packages]; D --> E[Files / JSON]; E --> F[Persistent application data];
```

Rule: Do not memorize import syntax blindly. Build small files, import them, inspect what Python is doing, deliberately break an import, and explain the result in your own words.

Connection to previous days: Functions gave you reusable behavior; collections gave you ways to model data. Modules now give you a way to organize those functions and data across files. Your Student Management System will become a persistent application by the end of Day 10.

PYTHON DAY 9

Modules, Packages, and Code Organization

1. Warm-up From Memory — 15 minutes

Without opening your previous project, explain on paper: What is a function? What is a variable? What is a dictionary? Why would a large program become difficult to maintain if every function lived in one huge file?

Then draw a possible Student Management System structure using separate responsibilities: students, reports, validation, and main menu.

2. What Is a Module? — 25 minutes

A module is a Python file that can contain definitions such as functions, classes, variables, and executable statements. The file name normally becomes the module name when imported.

Mental model: **one .py file = one reusable unit of Python code.**

```
# calculator.py
def add(a, b):
    return a + b

def multiply(a, b):
    return a * b
```

A different Python file can import the module and use its definitions:

```
# main.py
import calculator

print(calculator.add(10, 20))
print(calculator.multiply(5, 4))
```

Notice the dot: **calculator.add**. The module name identifies where the function came from.

3. Importing Modules — 30 minutes

Learn these forms and compare what name becomes available in your current program.

```
import math

from math import sqrt

import math as m

from math import sqrt as square_root
```

Form	Mental model
<code>import math</code>	Bring the module name into your namespace.
<code>from math import sqrt</code>	Bring a specific name from the module.
<code>import math as m</code>	Use an alias for the module name.
<code>from math import sqrt as square_root</code>	Import a name under another name.

Prediction experiment: before running an import example, predict which names you can type directly and which require a module prefix.

4. Experiment — Build Your First Module — 35 minutes

Create two files in the same folder: **operations.py** and **app.py**. Put several small functions in **operations.py**. Import them into **app.py**. Change one function and run **app.py** again.

Research questions:

- Does importing a module execute the top-level statements inside that module?
- What happens if the module name is misspelled?
- What happens if the imported function name is misspelled?
- What happens when two files define functions with the same name?

5. Executing Modules as Scripts — 35 minutes

A Python file can be used as an imported module or executed directly as a script. The common pattern `if __name__ == "__main__":` lets a file behave differently depending on how it is used.

```
def greet():  
    print("Hello from the module")  
  
if __name__ == "__main__":  
    greet()
```

When the file is executed directly, Python gives the special variable `__name__` the value `"__main__"`. When imported, the module receives its module name instead.

Experiment:

- Run the file directly.
- Import it from another file.
- Add a print statement above the if block.
- Move the print statement inside the if block.
- Observe which statements execute in each situation.

6. The Module Search Path — 30 minutes

When you write `import something`, Python must locate that module. Python searches locations described by its module search path.

```
import sys  
  
for location in sys.path:  
    print(location)
```

Do not memorize the exact directories shown on your computer. Your task is to understand that Python has a search path and that the environment, current project, standard library, and installed packages can affect what can be imported.

Research challenge: create a module with a common name such as `random.py` in your project directory. Import `random` and investigate what happens. Rename your file afterward so it does not interfere with future programs.

7. “Compiled” Python Files — 20 minutes

Python may create bytecode files with the `.pyc` extension, commonly stored inside a directory named `__pycache__`. These files are implementation details used to make repeated execution more efficient; they are not your original source code.

Research task: import a local module, inspect the project directory, locate `__pycache__`, and explain what you observe. Do not manually edit the generated file.

8. Standard Modules — 45 minutes

Python includes a large standard library. You should learn how to discover useful modules instead of memorizing every module.

Module	Typical use
math	Mathematical functions and constants.
random	Pseudo-random choices and values.
datetime	Dates and times.
os	Operating-system related operations.
sys	Interpreter and runtime information.
json	Encoding and decoding JSON data.

Investigation lab:

```
import math
import random
import datetime
import os
import sys

print(math.pi)
print(random.randint(1, 10))
print(datetime.datetime.now())
print(os.getcwd())
print(sys.version)
```

Do not simply copy the output into your notes. Write what each module is responsible for and identify one real-world problem where you might use it.

9. The dir() Function — 25 minutes

`dir()` helps you inspect the names available on an object or module.

```
import math

print(dir(math))
print(dir("Python"))
```

The output can be large. The important skill is learning how to investigate an unfamiliar object. Use `dir()` together with official documentation rather than assuming every discovered name is useful.

Research challenge:

- Use `dir()` on a list.
- Use `dir()` on a dictionary.
- Use `dir()` on a string.
- Find three methods you recognize and three you do not.
- Look up one unfamiliar method in the Python documentation.

10. Packages — 40 minutes

A package organizes related modules into a directory structure. A useful mental model is: **module** → **file, package** → **folder of related modules**.

```
project/
  main.py
  school/
    __init__.py
    students.py
```

```
reports.py
```

A package lets a larger application separate responsibilities without putting everything into one file.

11. Importing * From a Package — 25 minutes

Wildcard imports such as **from package import *** can make code shorter, but they can also make it unclear where a name came from. Packages can define `__all__` to control which names are exported by wildcard import behavior.

Professional habit: prefer explicit imports when clarity matters. The goal is readable code, not the shortest import statement.

12. Intra-package References — 30 minutes

Modules inside the same package may need to use other modules in that package. Python supports absolute and relative import styles.

```
# absolute style
from school.students import find_student

# relative style inside the school package
from .students import find_student
```

The dot in a relative import means the current package context. Experiment with a two-module package and explain which imports work when you run the package correctly.

13. Mini Project — Modular Student System — 60 minutes

Take your existing Student Management System and redesign its structure on paper before coding.

```
student_app/
  main.py
  students.py
  validation.py
  reports.py
  data/
    __init__.py
```

Suggested responsibilities:

- `students.py` → add, find, update, remove student records.
- `validation.py` → validate IDs, marks, attendance, and required fields.
- `reports.py` → average marks, performance, unique skills, and subject reports.
- `main.py` → menu, user interaction, and coordination.

Architecture challenge: keep each module focused. If a function starts doing validation, file storage, menu printing, and reporting at the same time, stop and decide which responsibility it belongs to.

14. Debugging Lab — 30 minutes

Run each problem separately. Identify the cause before fixing it.

```
# A
import maths

# B
from math import square_root

# C
import student_tools
print(student_tools.find_student(10))

# D
# In a project, create a file named random.py.
import random
print(random.randint(1, 10))
```

For each problem record:

- Exception name or unexpected behavior.
- Exact line responsible.
- What Python was trying to find.
- Why the import failed or behaved unexpectedly.
- The cleanest fix.

15. Main Project — Package-Based Expense System — 75 minutes

Use your earlier Employee Expense Reimbursement project as the architecture exercise. Design the project so that business rules, validation, reporting, and user interaction are not mixed together.

```
expense_app/
    main.py
    claims.py
    validation.py
    policies.py
    reports.py
```

Before writing code, document which functions belong in each module and why. Then build one module at a time and test it.

Knowledge Check

- Q1. What is a module?
- Q2. What is the difference between importing a module and importing a name from a module?
- Q3. What does `__name__` represent?
- Q4. Why is `if __name__ == "__main__":` useful?
- Q5. What is `sys.path`?
- Q6. What is `__pycache__` used for?
- Q7. What does `dir()` help you investigate?
- Q8. What is the difference between a module and a package?
- Q9. Why can wildcard imports reduce readability?
- Q10. What is a relative import?

Completion Checklist

- I can create and import my own module.

- I understand import, from ... import ..., and aliases.
- I can explain the `__main__` pattern.
- I investigated `sys.path`.
- I know what `__pycache__` is.
- I can use standard-library modules.
- I can use `dir()` to investigate an object.
- I understand the purpose of packages.
- I completed the modular Student Management System design.
- I completed the package-based Expense System architecture exercise.

PYTHON DAY 10

Input, Output, File Handling, and JSON

1. Warm-up From Memory — 15 minutes

Explain the difference between printing data and storing data. If a program prints a student's information and then closes, where does that information live? What must happen if the program should remember the student tomorrow?

2. Fancier Output Formatting — 30 minutes

Readable output becomes important as soon as programs display reports, invoices, logs, or tables. Python provides several ways to control how values appear.

3. Formatted String Literals — f-strings — 35 minutes

```
name = "Asha"
marks = 87.456

print(f"Student: {name}")
print(f"Marks: {marks:.2f}")
print(f"{{name:<15}} {{marks:>8.2f}}")
```

Useful formatting ideas:

- {value} inserts a value.
- {value:.2f} displays a floating-point value to two decimal places.
- {name:<15} gives a field a width and aligns text to the left.
- {marks:>8.2f} combines width, right alignment, and precision.

Prediction exercise: calculate the exact spacing before running the final line. Then change the field widths and observe how the report changes.

4. The String format() Method — 30 minutes

```
name = "Ravi"
score = 91.25

print("Student: {}".format(name))
print("Student: {0}, Score: {1:.2f}".format(name, score))
print("Student: {name}, Score: {score:.2f}".format(
    name=name, score=score
))
```

Compare format() with f-strings. The goal is not to declare one method 'good' and another 'bad'; learn to read all common forms because you will encounter them in existing Python code.

5. Manual String Formatting — 20 minutes

Manual formatting means deliberately constructing strings using operations such as concatenation, spaces, or padding. It can be useful for simple cases, but becomes harder to maintain when formatting rules grow.

```
name = "Asha"
score = 87

line = "Student: " + name + " | Score: " + str(score)
print(line)
```

Research question: when does manual concatenation become harder to read than an f-string? Give one example from a report you might build.

6. Old String Formatting — 20 minutes

```
name = "Ravi"  
score = 91.25  
  
print("Student: %s" % name)  
print("Score: %.2f" % score)  
print("%s scored %.2f" % (name, score))
```

The % style is older but still appears in existing code. Learn to recognize it even if you normally prefer newer formatting approaches.

7. Reading and Writing Files — 45 minutes

A file gives a program a way to store or retrieve information outside the program's running memory. The basic operation starts with `open()`.

```
file = open("notes.txt", "r")
content = file.read()
print(content)
file.close()
```

A safer modern pattern is the context manager:

```
with open("notes.txt", "r") as file:
    content = file.read()

print(content)
```

The **with** block manages the file resource so you do not have to remember to close it manually.

8. File Modes — 25 minutes

Mode	Purpose
r	Read existing content.
w	Write; creates or replaces the file.
a	Append to the end of a file.
x	Create a new file and fail if it already exists.
b	Binary mode modifier.
+	Update/read-write modifier.

Important: opening a file with **w** can replace existing content. Test this with a disposable file before using it on important data.

9. Methods of File Objects — 40 minutes

Method	Purpose
read()	Read content from the current position.
readline()	Read one line.
readlines()	Read lines into a list.
write()	Write a string.
writelines()	Write a sequence of strings.
seek()	Move the file position.
tell()	Report the current file position.
close()	Close the file resource.

Experiment with a small text file. Call `read()`, then investigate what happens when you call `read()` again without moving the file position.

10. Iterating Over a File — 25 minutes

```
with open("server.log", "r") as file:
    for line in file:
        print(line.strip())
```

This pattern is especially useful for log files because you can process one line at a time rather than loading the entire file into memory.

11. Practical Project — Production Log Analyzer — 60 minutes

Build on your previous log-analyzer work. Create a sample log file containing ERROR, WARNING, and INFO records. Read the file and generate a summary.

Required analysis:

- Total number of log lines.
- Number of ERROR records.
- Number of WARNING records.
- Number of INFO records.
- Most frequently appearing service.
- All critical/error lines.
- Search for a service entered by the user.
- Write the final summary to a separate report file.

Do not start by writing the entire program. Build it in stages: read → count → search → summarize → save.

12. Saving Structured Data With JSON — 50 minutes

Text files are excellent for human-readable logs and reports. JSON is useful when the program needs to preserve structured data such as dictionaries, lists, configuration, or records.

Mental model:

Python object → JSON representation → file → JSON representation → Python object

The standard library provides the `json` module for this conversion.

13. Python and JSON Data Types

Python	JSON
dict	object
list / tuple	array
str	string
int / float	number
True / False	true / false
None	null

14. `json.dumps()` and `json.loads()` — 30 minutes

```
import json

student = {
    "id": 101,
    "name": "Asha",
    "skills": ["Python", "SQL"]
}

text = json.dumps(student, indent=4)
print(text)

restored = json.loads(text)
print(restored)
```

`dumps()` converts a Python object into a JSON string. **`loads()`** converts a JSON string back into a Python object.

Prediction experiment: after `loads()`, inspect the type of the restored value. Then inspect the type of the skills field.

15. `json.dump()` and `json.load()` — 35 minutes

The file-oriented versions are useful when you want to store JSON directly in a file.

```
import json

student = {
    "id": 101,
    "name": "Asha",
    "skills": ["Python", "SQL"]
}

with open("student.json", "w") as file:
    json.dump(student, file, indent=4)

with open("student.json", "r") as file:
    restored = json.load(file)

print(restored)
```

Mental distinction:

- dumps / loads → work with JSON strings.
- dump / load → work directly with files.

16. JSON Debugging Lab — 25 minutes

```
import json

data = {"name": "Asha", "marks": {1, 2, 3}}
json_text = json.dumps(data)
print(json_text)
```

Investigate why this fails. Do not immediately search for a workaround. First ask: which Python value is being converted, and does JSON have a direct representation for it?

Then test other values such as a set, tuple, dictionary, list, None, True, and a nested dictionary. Record which conversions succeed.

17. Main Project — Persistent Student Management System — 90 minutes

Upgrade your existing Student Management System so that the program can preserve student records between executions.

Target data model:

```
{
  "101": {
    "name": "Asha",
    "email": "asha@example.com",
    "subjects": ["Maths", "Physics"],
    "marks": {
      "Maths": 85,
      "Physics": 78
    },
    "attendance": 91,
    "skills": ["Python", "SQL"],
    "active": True
  }
}
```

Required architecture:

- On startup, attempt to load students.json.
- If there is no saved data yet, start with an empty data structure.
- Add a student and save the updated data.
- Update a student and save the updated data.
- Delete a student and save the updated data.
- View/search students from the in-memory structure.
- Generate statistics from the loaded data.
- Before exiting, ensure changes are persisted.

Architecture challenge:

Keep responsibilities separate. One part of the program should deal with storage, while other functions should focus on student operations. This prepares you for larger applications where storage may later change from JSON to a database.

Suggested structure:

```
student_app/
  main.py
  students.py
  storage.py
  reports.py
  students.json
```

18. JSON + Your Expense Reimbursement Project — 45 minutes

Apply the same persistence idea to your Employee Expense Reimbursement system. Decide what information should survive after the program closes: employees, submitted claims, approval status, reimbursement totals, and rejection reasons.

Research/design questions:

- What should be stored as a list and what should be stored as a dictionary?
- What should uniquely identify a claim?

- When should the program save?
- What happens if the JSON file is empty or malformed?
- What happens if a saved record is missing a field your current program expects?

19. Integrated Challenge — File + JSON + Modules

Design a small application that combines everything learned in Days 9 and 10.

Example: CLI Service Incident Tracker

- Use modules for incident operations, validation, reports, and storage.
- Store structured incidents in JSON.
- Export a human-readable report to a text file.
- Use f-strings or format() for the report.
- Load the data when the program starts.
- Save changes after create/update operations.
- Use a main guard in the entry-point module.

20. Research Challenge — What Changes When JSON Becomes a Database?

Your JSON application works for a learning project, but larger systems often need databases. Research the differences between JSON-file persistence and a database. Focus on searching, concurrency, validation, transactions, and scale.

Knowledge Check

- Q1. What is the difference between print output and persistent storage?
- Q2. What does open() do?
- Q3. What is the difference between r, w, and a?
- Q4. Why is with open(...) useful?
- Q5. What does read() return?
- Q6. What do seek() and tell() represent?
- Q7. What is JSON?
- Q8. What is the difference between dumps() and dump()?
- Q9. What is the difference between loads() and load()?
- Q10. What happens to True, None, lists, and dictionaries when represented in JSON?
- Q11. Why might JSON be useful for a small CLI application's persistence?
- Q12. Why might a database be preferable for a larger multi-user application?

Completion Checklist

- I can format output with f-strings.
- I can read and understand format() examples.
- I recognize old % formatting.
- I can open, read, write, append, and close files.
- I understand why context managers are useful for files.

- I can use `read()`, `readline()`, `readlines()`, `write()`, and iteration.
- I understand JSON as structured data representation.
- I can use `dumps/loads` and `dump/load`.
- I completed the persistent Student Management System.
- I designed persistence for the Expense Reimbursement project.
- I completed the integrated Modules + Files + JSON challenge.

Final Review — Days 9 & 10

The Learning Cycle

LEARN ↓ CODE ↓ PREDICT ↓ BREAK IT ↓ INVESTIGATE ↓ FIX IT ↓ BUILD ↓ EXPLAIN

Final practical test — complete without looking at this document:

- Create a two-file Python program where one file imports a function from another.
- Use a main guard correctly.
- Inspect `sys.path` and explain why a module can or cannot be found.
- Create a package with two related modules.
- Read a text log and produce counts for `ERROR` and `WARNING`.
- Write the report to a file using readable formatting.
- Save a nested Python dictionary to JSON.
- Load it back and confirm the structure.
- Explain the difference between JSON strings and JSON files.
- Explain why modular design is easier to maintain than one huge file.

Suggested Study Schedule

Day 9: 20 min warm-up | 30 min modules | 35 min main guard | 30 min search path | 20 min bytecode | 45 min standard modules + `dir()` | 40 min packages | 30 min debugging | 60–75 min architecture project.

Day 10: 15 min warm-up | 35 min f-strings | 30 min `format()` | 20 min old formatting | 45 min file handling | 50 min JSON | 25 min debugging | 90 min persistent Student Management System | 45 min integrated challenge.

Official References

Python Documentation — <https://docs.python.org/3/>

Python Tutorial — <https://docs.python.org/3/tutorial/>

The Python Standard Library — <https://docs.python.org/3/library/>

Python JSON documentation — <https://docs.python.org/3/library/json.html>

Course philosophy: learn fundamentals deeply, build something practical, investigate behavior instead of memorizing it, and gradually develop the habits needed to read technical documentation and research unfamiliar systems.